

课程学习小组讨论记录

【讨论主题】：AI 对软件开发的影响及工具分享

【课程名称】：软件项目开发与实践

【讨论日期】：2026 年 6 月

【讨论时长】：2 小时

【主持人】：谢宇鹏

【参与人员】：谢宇鹏（学号：5120243390）

第 1 部分 讨论背景与目标

【讨论背景】

近年来，AI Coding 工具经历了从「代码补全」到「智能协作」的跨越式发展。2022 年 GitHub Copilot 的发布标志着 AI 辅助编程进入实用阶段，而 2024-2025 年间 Cursor、Claude Code、Trae 等工具的涌现，则将 AI 编程推向了「Agent 模式」——AI 不再仅仅是补全工具，而是能够理解需求、规划架构、编写完整模块、调试修复的智能协作伙伴。

课程中采用 B 模式（AI 智能协作开发），要求学生在 AI 辅助下完成游戏项目开发。这要求我们不仅要会使用 AI 工具，更要理解 AI 的能力边界，学会「驾驭」而非「被驾驭」。本项目「星露谷战斗模块」正是在这一背景下，以 Python + Pygame 为技术栈，在 AI 辅助下完成从需求分析到代码实现的全流程。

本次讨论旨在：深入分析 AI 对软件开发各环节的影响，分享本项目中使用的 AI 工具和实际经验，讨论 AI 辅助开发中的交互技巧和迭代策略，总结对软件工程专业学生职业发展的启示。

【讨论目标】

1. 系统分析 AI 对软件开发各环节（需求、设计、编码、测试、维护）的影响
2. 对比分析本项目中使用的 AI 开发工具的特点和适用场景
3. 总结 AI 辅助开发中的有效交互策略和迭代方法
4. 讨论「提示词工程」在 B 模式开发中的重要性
5. 总结 AI 时代软件工程师的角色转变和职业发展启示

第 2 部分 主要讨论内容

AI 对软件开发各环节的影响

需求分析阶段

- 积极影响：AI 可以根据模糊的想法（如「做一个类似星露谷的战斗游戏」）快速展开为结构化的需求列表，帮助梳理功能点、用户群体、场景设计
- 积极影响：AI 能够辅助生成需求文档框架，提供专业的软件工程术语和文档结构
- 潜在问题：AI 对「游戏性」的理解可能偏技术化，缺乏对玩家体验的感性判断
- 潜在问题：AI 生成的需求可能过于宽泛，需要人工裁剪和细化边界
- 本项目实践：在本项目中，先与 AI 讨论游戏定位（「星露谷的战斗模块」），再让 AI 细化功能点（14 个功能需求），最后人工调整优先级和实现细节

架构设计阶段

- 积极影响：AI 能够根据需求快速生成合理的模块划分、类结构设计、数据流图
- 积极影响：AI 可以对比多种架构方案（如 MVC、ECS、分层架构），分析优劣
- 潜在问题：AI 生成的架构可能「过度设计」，引入不必要的复杂度
- 潜在问题：AI 可能忽略项目的实际约束（如 Pygame 的 API 限制、课程的时间限制）
- 本项目实践：向 AI 描述期望的架构风格（「分层架构，面向对象」），让 AI 生成类结构图，再人工审查和简化

编码实现阶段

- 积极影响：AI 可以快速生成样板代码（如精灵类、状态机、碰撞检测框架），大幅提高编码效率
- 积极影响：AI 对 Pygame API 的熟悉程度很高，可以快速生成正确的 API 调用
- 积极影响：遇到 bug 时，将错误信息反馈给 AI，AI 可以快速定位和修复
- 潜在问题：AI 生成的代码可能有逻辑错误（如碰撞检测的边界情况）
- 潜在问题：AI 可能生成「能运行但不优雅」的代码，需要人工重构
- 本项目实践：逐模块生成代码（先地图系统 → 再玩家 → 再敌人 → 再武器），每生成一个模块立即运行测试

测试与调试阶段

- 积极影响：AI 可以根据功能需求自动生成测试用例
- 积极影响：AI 可以根据错误堆栈快速定位问题，提供修复建议
- 潜在问题：AI 的「修复」可能引入新的 bug，需要验证
- 本项目实践：将运行时错误信息复制给 AI，AI 给出修复方案，验证后应用

AI 开发工具详细对比

本项目中使用和评估了以下 AI 开发工具：

工具	类型	本项目用途	优点	缺点
Claude Code	CLI 工具	需求讨论、架构设计、代码生成、文档撰写	长文本理解能力强，适合深度讨论；代码生成质量高；支持 Agent 模式直接修改文件	需要 API 费用
GitHub Copilot	IDE 插件	日常编码辅助、代码补全	与 IDE 集成流畅，对 Pygame API 支持好，补全响应快	仅补全，无法主动规划
Cursor	AI IDE	全流程开发、文件编辑	体验流畅，支持 Agent 模式，可直接创建/修改多个文件	学习成本
Trae	国产 AI IDE	备选方案	免费使用，中文支持好，功能逐步完善	免费版有次数限制
ChatGPT	网页对话	技术调研、算法解释	灵活，知识面广	非实时，代码上下文有限

工具选型建议：

- 深度讨论和架构设计：推荐 Claude Code，长文本理解能力强，适合需求讨论
- 日常编码辅助：推荐 GitHub Copilot 或 Cursor，IDE 集成体验好
- 全流程开发：推荐 Cursor 或 Claude Code 的 Agent 模式，可直接操作文件
- 预算有限：推荐 Trae（免费）+ ChatGPT（免费版）

AI 交互策略与迭代方法

需求阶段的交互

1. **从模糊到精确**：先给出宽泛描述（「做一款类似星露谷的战斗游戏」），再逐步细化（「要有哪些武器？几种怪物？几张地图？」）
2. **让 AI 展开**：告诉 AI「请帮我列出完整的功能需求清单」，AI 会生成结构化列表
3. **人工裁剪**：AI 生成的需求可能过多，需要根据时间和能力裁剪优先级
4. **文档化**：将讨论结果保存为文档，作为后续开发的输入

设计阶段的交互

1. **指定约束**：明确告诉 AI 技术栈（「Python + Pygame」）、架构风格（「面向对象，多文件」）、限制条件（「不用 TMX 格式」）
2. **要求解释**：让 AI 解释为什么选择某种架构或算法，而不仅仅是给出代码
3. **对比方案**：让 AI 列出多种方案并对比优劣，再做决策
4. **生成文档**：让 AI 生成设计文档框架，补充技术细节

编码阶段的交互

1. **逐模块生成**：不要一次性生成整个项目，而是逐模块（地图 → 玩家 → 敌人 → 武器）逐步生成
2. **提供上下文**：生成新模块时，告诉 AI 已有的类结构和接口，确保兼容
3. **立即验证**：每生成一个模块立即运行测试，发现问题立即反馈给 AI 修复
4. **保留版本**：每次 AI 成功生成的代码立即保存（git commit），便于回溯

调试阶段的交互

1. **提供完整错误信息**：将完整的错误堆栈复制给 AI，而非只描述症状
2. **描述期望行为**：告诉 AI「我期望 X 行为，但实际发生了 Y 行为」
3. **验证修复**：AI 给出修复方案后，先理解再应用，而非盲目复制
4. **回归测试**：修复一个 bug 后，验证其他功能是否受影响

提示词工程 (Prompt Engineering)

在 B 模式开发中，提示词的质量直接影响 AI 输出的质量。总结以下技巧：

好的提示词特征

1. **明确目标**：「请用 Python 实现一个 A* 寻路算法」而非「帮我写个寻路」
2. **指定约束**：「在 20×11 的网格上，使用曼哈顿距离启发函数」
3. **提供上下文**：「我正在开发一款 Pygame 游戏，已有 Entity 基类包含 move() 和 collision() 方法」
4. **要求格式**：「请返回完整的 Python 代码，包含注释」
5. **迭代优化**：「这段代码可以运行，但请优化性能，减少每帧的计算量」

常见提示词误区

1. **过于模糊**：「帮我做个游戏」——AI 不知道做什么类型、什么规模
2. **缺乏约束**：「写个 AI」——没有指定语言、框架、具体功能
3. **一次太多**：「帮我做整个项目」——应该拆分为多个小任务
4. **不给反馈**：AI 第一次输出不满意时，应描述哪里不满意，而非重新提问

AI 时代开发者角色转变

从「写代码」到「审查代码」

传统开发中，开发者的核心工作是编写代码。AI 时代，代码产出不再是核心竞争力——AI 可以在几秒钟内生成开发者几小时才能写出的代码。开发者的角色转向「审查者」：判断 AI 输出的质量，识别潜在问题，确保代码符合项目需求。

从「实现功能」到「设计系统」

AI 擅长实现具体功能（如「实现一个碰撞检测」），但不擅长设计整体架构（如「如何划分模块使系统可扩展」）。架构设计能力成为开发者的核心壁垒。

从「掌握语法」到「理解原理」

AI 可以写出语法正确的代码，但开发者需要理解底层原理（如 A* 算法为什么选择曼哈顿距离、分轴碰撞为什么先水平后垂直），才能判断 AI 输出的合理性。

提示词工程成为新技能

能够精准描述需求、分解任务、引导 AI 输出高质量结果，成为 AI 时代开发者的核心技能之一。这与传统的需求分析能力一脉相承，但要求更高——不仅要理解业务，还要理解 AI 的能力边界。

第 3 部分 总结与反思

讨论总结

通过本次深入讨论，我们得出以下核心结论：

1. **AI 是工具不是替代：**AI 大幅提高了开发效率，但不能替代开发者的思考和判断
2. **需求能力更重要：**「做什么」比「怎么做」更有价值，需求分析和架构设计成为核心竞争力
3. **基础是前提：**数据结构、算法、设计模式等基础知识是驾驭 AI 工具的前提，缺乏基础将无法判断 AI 输出的合理性
4. **交互是关键：**有效的提示词和迭代策略决定了 AI 辅助开发的质量和效率
5. **审查是底线：**所有 AI 生成的代码都必须经过人工审查，确保质量和安全

对软件工程专业学生的启示

1. **夯实基础：**数据结构、算法、操作系统、计算机网络等基础知识是不可替代的专业壁垒
2. **培养架构思维：**从「实现功能」转向「设计系统」，理解模块划分、依赖管理、扩展性设计
3. **掌握提示词工程：**学会精准描述需求、分解任务、引导 AI 输出高质量结果
4. **建立审查能力：**能够判断 AI 输出的质量，识别潜在问题并修正
5. **拥抱变化：**AI 不会取代软件工程师，但会取代不会使用 AI 的软件工程师
6. **持续学习：**AI 工具在快速迭代，保持学习和适应能力是长期竞争力